

Sistema de Registro Escolar ClassFlow

Entrega Final

Angelo Millan
Evens Reneus

Desarrollo Fullstack 3

Julio, 2026

1. Resumen Ejecutivo

ClassFlow es un sistema de gestión educativa desarrollado con arquitectura de microservicios, diseñado para centralizar y automatizar procesos académicos como administración de usuarios, cursos, evaluaciones, asistencia, mensajería y notificaciones. El proyecto está compuesto por un frontend en React 18 + TypeScript + Vite y un backend distribuido en 7 microservicios Java (Spring Boot 3.5.14, JDK 25), orquestados localmente con Docker Compose y desplegados en Kubernetes. Cada microservicio gestiona su propia base de datos PostgreSQL con migraciones versionadas mediante Flyway, y se comunican a través de un API Gateway (Spring Cloud Gateway) y un BFF (Backend-for-Frontend) reactivo. El frontend implementa portales diferenciados por rol (administrador, docente, estudiante y apoderado) con autenticación basada en JWT. El proyecto incluye datos semilla para usuarios, cursos (1° Básico a 4° Medio), evaluaciones y asistencias, cobertura de pruebas unitarias y documentación de APIs mediante Swagger/OpenAPI en cada servicio.

2. Arquitectura del Sistema

ClassFlow sigue una arquitectura de microservicios con separación estricta en capas (presentación, negocio, datos). Cada microservicio es independiente, con su propia base de datos, y la comunicación entre ellos ocurre exclusivamente vía REST. El frontend es una SPA en React que se comunica con el backend a través de un API Gateway y un BFF.

2.1. Diagrama C1 (Contexto)

El diagrama C1 muestra a ClassFlow como un sistema central con el que interactúan cuatro tipos de actores: el Administrador, que gestiona usuarios, cursos y la configuración general del sistema; el Docente, encargado de registrar asistencia, calificaciones y anotaciones sobre los estudiantes; el Estudiante, que consulta sus notas, asistencias y evaluaciones pendientes; y el Apoderado, que supervisa el rendimiento académico y la asistencia de sus pupilos. Los cuatro actores acceden al sistema exclusivamente a través del frontend web, sin comunicación directa con el backend ni con las bases de datos. El sistema se representa como una única caja negra que abstrae toda la complejidad interna (microservicios, bases de datos, gateway, BFF) y expone hacia el exterior únicamente las funcionalidades de autenticación, gestión académica, asistencia, mensajería y notificaciones. Este nivel de abstracción es el punto de partida del modelo C4: define el alcance del sistema y quiénes se relacionan con él, sin entrar en detalles de implementación.

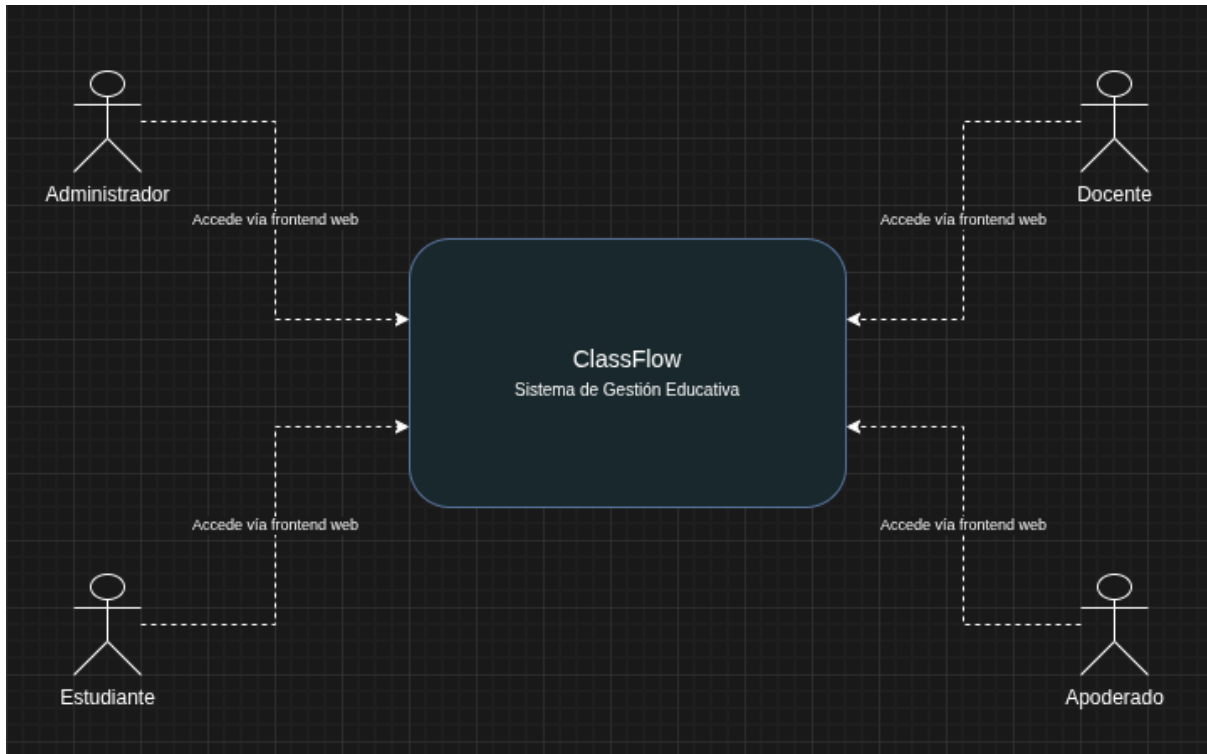


Fig 1. Diagrama C1. Es una representación visual de cómo las personas interactúan con el sistema, siendo su único punto de entrada el Frontend.

2.2. Diagrama C2 (Contenedores)

El diagrama C2 (Contenedores) descompone ClassFlow en los contenedores que ejecutan el sistema. En la parte superior se ubican los cuatro actores (Administrador, Docente, Estudiante y Apoderado), quienes acceden únicamente al Frontend, una SPA construida con React 18, TypeScript y Vite, servida a través de Nginx en el puerto 3000. Todas las peticiones del frontend se dirigen al API Gateway (Spring Cloud Gateway, puerto 8080), que actúa como único punto de entrada al backend. El gateway enruta las solicitudes según el path: `/api/bff/*` hacia el BFF (Backend-for-Frontend, Spring WebFlux, puerto 8086), que agrega datos de múltiples fuentes para construir las respuestas del dashboard; y `/api/auth/*`, `/api/courses/*`, `/api/attendance/*`, `/api/messages/*` y `/api/notifications/*` hacia cada uno de los cinco microservicios (ms-auth, ms-academic, ms-assistance, ms-message, ms-notification). Cada microservicio es una aplicación Spring Boot 3.5.14 con JDK 25, y cada uno posee su propia base de datos PostgreSQL 15 dedicada (auth-db, academic-db, assistance-db, message-db y notification-db), con las que se comunican vía JDBC. No existen conexiones directas entre bases de datos de diferentes servicios. El diagrama refleja así la separación de responsabilidades y el aislamiento de datos que exige el patrón Database per Service.

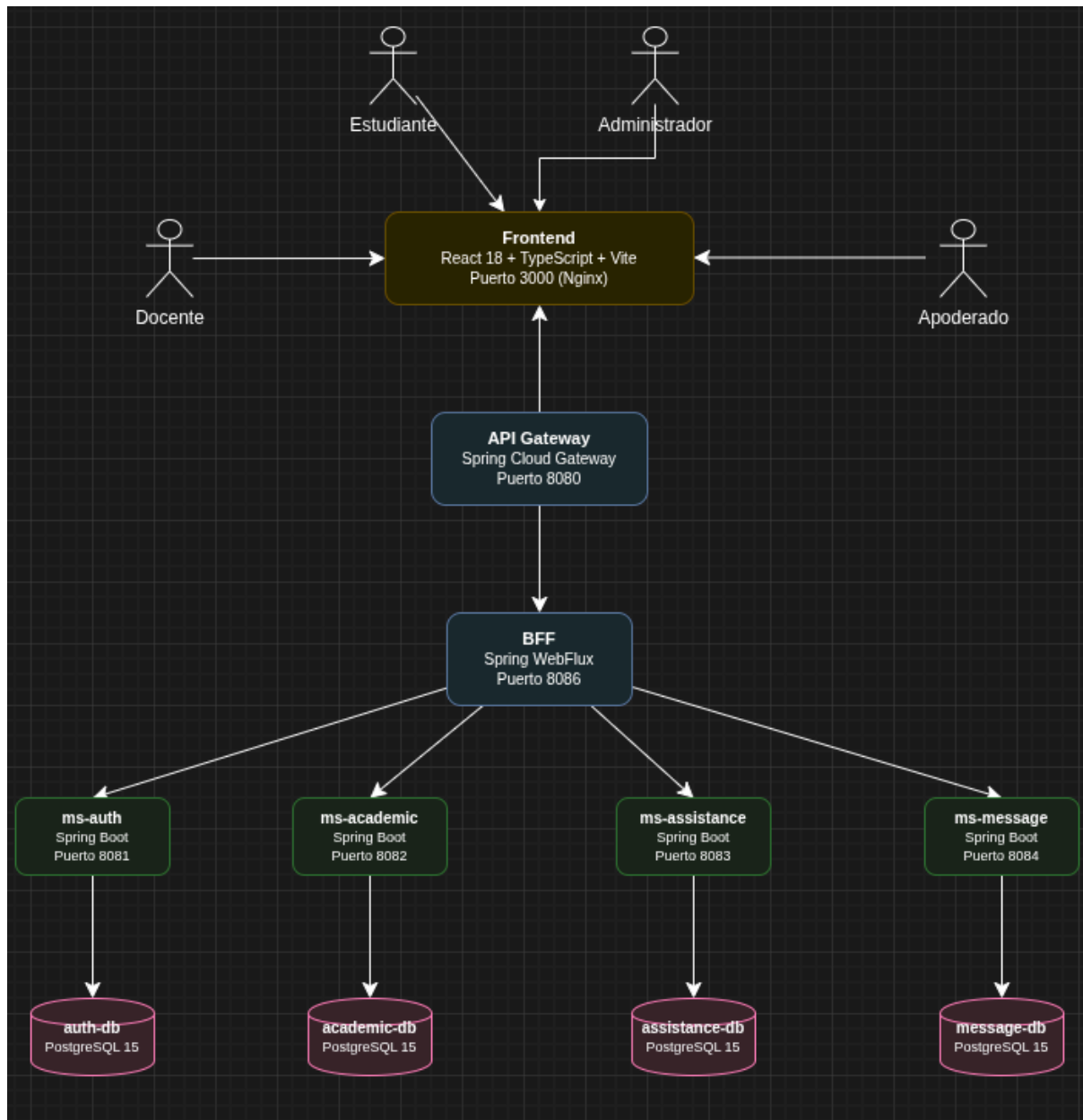


Fig 2. Diagrama C2. Esta figura grafica la estructura del sistema con sus componentes principales, y también la lógica de recorrido de las peticiones. desde el Frontend.

2.3. Diagrama C3 (Componentes)

El diagrama C3 (Componentes) toma como referencia el microservicio ms-academic para mostrar la estructura interna que todos los microservicios del proyecto replican. Está organizado en tres capas verticales. La capa de presentación contiene cuatro controladores REST (CourseController, SubjectController, EvaluationController y GradeController), cada uno anotado con @RestController y su respectivo @RequestMapping. Estos controladores reciben las peticiones HTTP, delegan la lógica a la capa inferior y devuelven respuestas en formato DTO. La capa de negocio está compuesta por los servicios correspondientes (CourseService, SubjectService, EvaluationService y GradeService), anotados con @Service, donde se implementa la lógica de negocio y la orquestación de llamadas a

repositorios. La capa de datos incluye los repositorios JPA (CourseRepository, SubjectRepository, EvaluationRepository y GradeRepository), que extienden JpaRepository y proporcionan acceso a la base de datos mediante Hibernate, y las entidades JPA (Course, Subject, Evaluation y Grade), que mapean las tablas de la base de datos PostgreSQL. De forma transversal, el GlobalExceptionHandler (@RestControllerAdvice) captura excepciones de cualquier capa y devuelve respuestas JSON estandarizadas; el SwaggerConfig expone la documentación de los endpoints vía OpenAPI; y los DTOs (CourseDTO, SubjectDTO, EvaluationDTO, GradeDTO) definen los contratos de entrada y salida entre el controller y el cliente. Cada microservicio sigue exactamente este mismo patrón, variando solo las entidades y la lógica de negocio según su dominio.

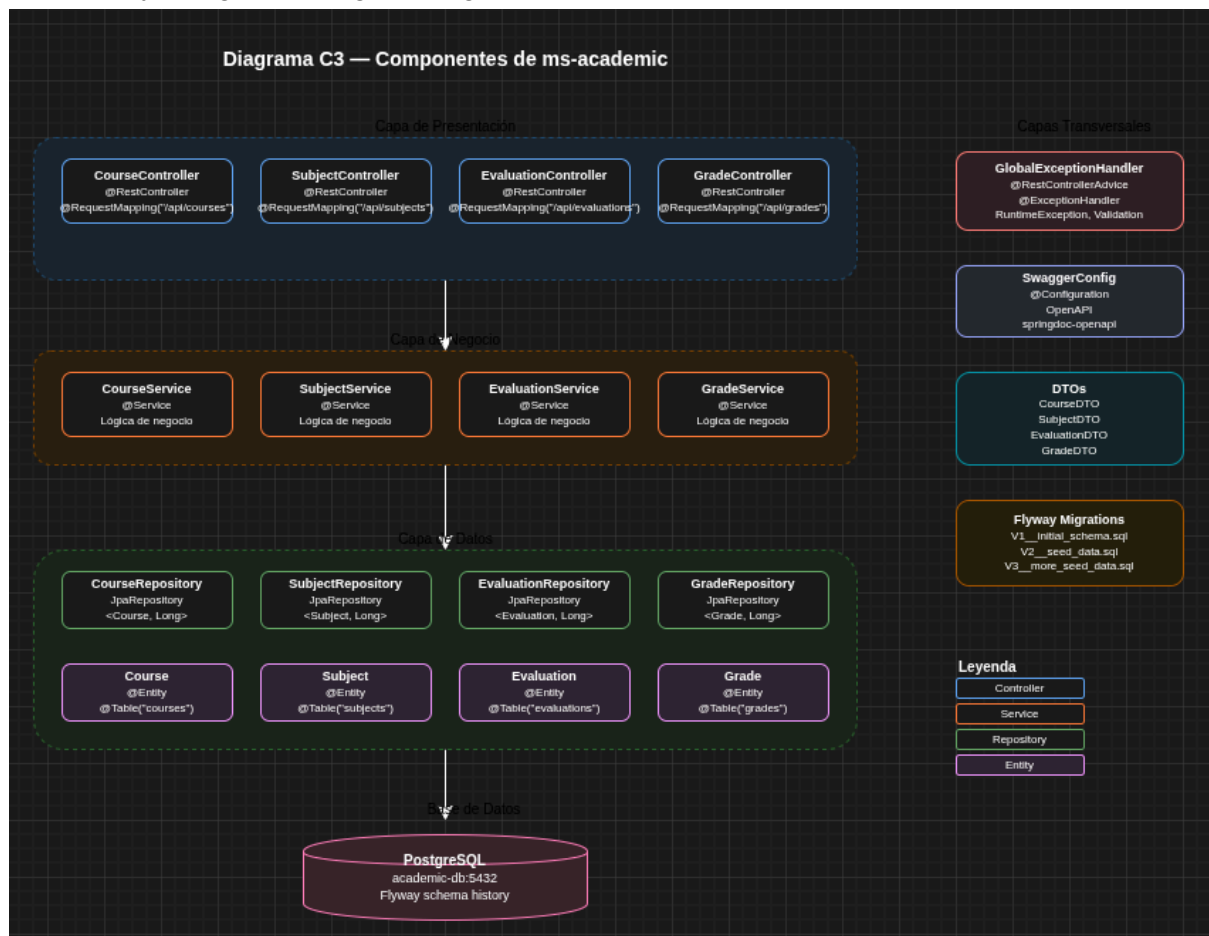


Fig 3. Diagrama C3. Muestra el detalle de la estructura del microservicio ms-auth. Muestra la ruta interna que las peticiones siguen desde que son ingresadas a través de los endpoints expuestos en el Controller, las comprobaciones de negocio en la capa de Servicio, hasta las llamadas a la base de datos en Repository.

3. Estructura del Repositorio

El proyecto sigue una estructura de directorios plana y predecible, organizada en tres grandes carpetas en la raíz: backend, frontend y /docs.

```

classflow/
├── backend/
│   ├── docker-compose.yml          # Orquestación local (14 contenedores)
│   ├── .env                        # Variables de entorno (JWT, API keys, etc.)
│   └── README.md
│
│   ├── api-gateway/                # Spring Cloud Gateway (puerto 8080)
│   │   ├── Dockerfile, pom.xml
│   │   └── src/main/java/com/ohiggins/classflow/gateway/
│   │       ├── GatewayServiceApplication.java
│   │       ├── config/              # CorsConfig, SwaggerConfig
│   │       ├── security/            # SecurityConfig (filtro JWT)
│   │       └── exception/           # GlobalExceptionHandler, ErrorResponse
│   │
│   ├── bff/                        # Backend-for-Frontend (puerto 8086)
│   │   ├── Dockerfile, pom.xml
│   │   └── src/main/java/com/ohiggins/classflow/bff/
│   │       ├── BffServiceApplication.java
│   │       ├── controller/          # DashboardController
│   │       ├── service/             # DashboardService
│   │       ├── dto/                 # DashboardResponse
│   │       ├── config/              # WebClientConfig, SwaggerConfig
│   │       ├── security/            # SecurityConfig (filtro JWT)
│   │       └── exception/           # GlobalExceptionHandler, ErrorResponse
│   │
│   ├── ms-auth/                    # Autenticación y usuarios (puerto 8081)
│   │   ├── Dockerfile, pom.xml
│   │   └── src/main/java/com/ohiggins/classflow/auth/
│   │       └── controller/          # AuthController (login, register,
│   │           validate...)
│   │           └── service/          # AuthService, UserService,
│   │               PasswordResetService
│   │               ├── repository/   # UserRepository
│   │               ├── entity/       # User, Role
│   │               ├── dto/          # LoginRequestDTO, LoginResponseDTO, etc.
│   │               ├── security/     # JwtTokenProvider, SecurityConfig
│   │               ├── config/       # SwaggerConfig
│   │               ├── exception/    # GlobalExceptionHandler, ErrorResponse
│   │               └── resources/db/migration/ # 6 migraciones Flyway
│   │
│   ├── ms-academic/                # Cursos, evaluaciones, notas (puerto 8082)
│   ├── ms-assistance/              # Asistencia y anotaciones (puerto 8083)
│   ├── ms-message/                 # Mensajes y anuncios (puerto 8084)
│   ├── ms-notification/            # Notificaciones (puerto 8085)
│   └── (misma estructura en capas: controller/service/repository/entity/

```

```

    |
    |         dto/security/config/exception)
    |
    |         └─ k8s/                                # Manifiestos de Kubernetes
    |             ├── namespace.yml, secret.yml, configmap.yml
    |             ├── api-gateway.yml, bff.yml, frontend.yml
    |             ├── ms-*.yaml                       # Un deployment por microservicio
    |             ├── ingress-traefik*.yaml
    |             └─ databases/                        # Manifiestos de las 5 bases de datos
    |
    | └─ frontend/                                   # React 18 + TypeScript + Vite
    |     ├── package.json, tsconfig.json, vite.config.ts
    |     ├── Dockerfile, nginx.conf
    |     └─ src/
    |         ├── pages/                             # Login, ForgotPassword, ResetPassword,
    |                                                     # AdminDashboard, TeacherAccountPage,
    |                                                     # StudentDashboardPage, GuardianDashboardPage
    |         ├── router/index.tsx                   # Rutas protegidas por rol
    |         ├── services/                          # api, auth, user, dashboard, course,
    |                                                     # grade, annotation, media
    |         ├── context/                           # AuthContext, ThemeContext
    |         ├── hooks/                             # useDashboardData, useFetch, useForm,
    |                                                     # useAsync, useLogout, useRawDashboard,
    |                                                     # useTeacherCourseDetail
    |         ├── components/                        # Button, Input, Loading, Error,
    |                                                     # ProtectedRoute, MediaImage, Layout
    |         ├── config/ constants/ types/ utils/ styles/
    |         └─ main.tsx
    |
    | └─ docs/                                       # Documentación del proyecto
    |     ├── arquitectura.md, microservicios.md, frontend.md
    |     ├── api.md, seguridad.md, infraestructura.md
    |     ├── pruebas.md, convenciones.md
    |     └─ diagramas/                             # Fuentes C1, C2, C3
    ...

```

Cada microservicio dentro de backend sigue la misma convención: su nombre empieza con ms- (excepto bff y api-gateway), incluye Dockerfile, pom.xml y README.md propios, y su código fuente se organiza bajo src/main/java/com/ohiggins/classflow/{artefacto}/ con las subcarpetas controller/, service/, repository/, entity/, dto/, config/, exception/. Las migraciones Flyway residen en src/main/resources/db/migration/. El frontend está íntegramente en TypeScript, con tipado estricto y alias de importación configurados en tsconfig.json. La carpeta /docs/ contiene los diagramas C4 en formato editable y los archivos PDF de presentación, informe técnico y caso de estudio.

4. Microservicios

ClassFlow está compuesto por 5 microservicios Java que se ejecutan como contenedores independientes. Cada uno es una aplicación Spring Boot 3.5.14 con JDK 25, packaging en JAR y despliegue mediante Docker. La siguiente tabla resume sus características técnicas:

Servicio	Artefacto	Package base	Puerto	BD propia	Rol principal
Auth	ms-auth	com.ohiggins.classflow.auth	8081	auth-db	Autenticación JWT, CRUD de usuarios con 4 roles
Academic	ms-academic	com.ohiggins.classflow.academic	8082	academic-db	Cursos, asignaturas, evaluaciones, calificaciones
Assistance	ms-assistance	com.ohiggins.classflow.assistance	8083	assistance-db	Asistencia diaria y anotaciones de estudiantes
Message	ms-message	com.ohiggins.classflow.message	8084	message-db	Mensajería entre usuarios y anuncios por curso
Notification	ms-notification	com.ohiggins.classflow.notification	8085	notification-db	Notificaciones push/email y alertas del sistema
BFF	bff	com.ohiggins.classflow.bff	8086	-	Agregación de datos para los dashboards del frontend
Gateway	api-gateway	com.ohiggins.classflow.gateway	8080	-	Enrutamiento, punto de entrada único, balanceo

Dependencias comunes a todos los servicios con base de datos:

Spring-boot-starter-web, spring-boot-starter-data-jpa, springdoc-openapi (Swagger/OpenAPI), flyway-core + flyway-database-postgresql, Lombok, PostgreSQL driver y H2 (para desarrollo local).

Casos particulares:

- ms-auth añade spring-boot-starter-security, jjwt (JWT) y BCryptPasswordEncoder.
- bff usa spring-boot-starter-webflux (reactivo) en vez de webmvc.

- api-gateway usa spring-cloud-starter-gateway (reactivo) y no tiene base de datos.

Comunicación entre servicios:

Toda ocurre vía REST síncrono. El frontend nunca se comunica directamente con los microservicios; lo hace siempre a través del API Gateway, que enruta según el path. El BFF, a su vez, consulta a los 5 microservicios de datos para consolidar las respuestas del dashboard. No existe comunicación directa entre microservicios de datos; si un servicio necesita información de otro, la solicita a través del gateway o mediante llamada REST directa si está configurada (como hace el BFF).

5. Patrón Database per Service

Cada microservicio con estado gestiona su propia base de datos PostgreSQL, sin compartir esquemas ni tablas con otros servicios. Esta decisión arquitectónica sigue el patrón Database per Service y se materializa en docker-compose.yml con 5 contenedores de base de datos independientes

Microservicio	Base de datos	Usuario	Volumen persistente
ms-auth	auth-db	auth-user	auth-postgres-data
ms-academic	academic-db	academic-user	academic-postgres-data
ms-assistance	assistance-db	assistance-user	assistance-postgres-data
ms-message	message-db	message-use	message-postgres-data
ms-notification	notification-db	notification-user	notification-postgres-data

Reglas del patrón aplicadas en el proyecto:

Aislamiento total:

- Ningún microservicio accede directamente a la base de datos de otro. La única forma de obtener datos de otro dominio es mediante llamadas REST a través del API Gateway o desde el BFF. No hay claves foráneas entre tablas de diferentes bases de datos, ni consultas entre servicios.

Migraciones versionadas con Flyway:

- Cada microservicio gestiona su propio esquema mediante archivos SQL versionados en src/main/resources/db/migration/. Al iniciar, Flyway compara la versión de la base de datos con los archivos disponibles y aplica las migraciones pendientes automáticamente. En total son 17 migraciones distribuidas entre los 5 servicios.

Dos perfiles de ejecución:

- En el perfil default (desarrollo local sin Docker), los servicios usan H2 en memoria para no depender de PostgreSQL instalado. En el perfil docker (el que se usa en docker compose up), se conectan a sus respectivos contenedores PostgreSQL con las credenciales definidas en las variables de entorno.

Volúmenes persistentes:

- Cada base de datos tiene su propio volumen Docker nombrado (ej. academic-postgres-data), lo que garantiza que los datos sobrevivan a reinicios del contenedor. Al ejecutar `docker compose down -v` los volúmenes se eliminan y las migraciones se re-ejecutan desde cero.

6. Frontend

El frontend de ClassFlow es una Single Page Application (SPA) construida con React 18, TypeScript 5.3 y Vite 8 como bundler. El proyecto se encuentra en frontend y sigue una estructura modular con separación clara de responsabilidades.

La aplicación utiliza react-router-dom para el enrutamiento del lado del cliente. Todas las rutas están protegidas mediante el componente ProtectedRoute, que verifica el rol del usuario autenticado antes de renderizar cada página. Las rutas se organizan bajo `/dashboard/{rol}`

Ruta	Componente	Rol Requerido
<code>/dashboard/admin</code>	AdminDashboard	ADMINISTRATOR
<code>/dashboard/teacher</code>	TeacherDashboard	TEACHER
<code>/dashboard/student</code>	StudentDashboard	STUDENT
<code>/dashboard/guardian</code>	GuardianDashboard	GUARDIAN
<code>/login</code>	LoginPage	Público
<code>/access-denied</code>	AccesDeniedPage	Todos

Conexión con el backend.

El frontend nunca se comunica directamente con los microservicios. Todas las peticiones se realizan a través del API Gateway (`http://localhost:8080/api`) usando axios como cliente HTTP. El flujo típico es: el frontend solicita `GET /api/auth/login` al gateway, este lo enruta a ms-auth, y la respuesta viaja de vuelta por el mismo camino. Para los dashboards, el frontend llama a `GET /api/bff/dashboard/{userId}` que el gateway enruta al BFF, el cual consolida datos de los 5 microservicios y devuelve una respuesta única.

Autenticación.

El login envía credenciales al endpoint `/api/auth/login`, que devuelve un token JWT. El token se almacena en `localStorage` y se envía en el header `Authorization: Bearer <token>` en cada petición posterior. El `AuthContext` mantiene el estado global del usuario autenticado y expone funciones como `login()`, `logout()` y `validate()`.

Portales por rol.

Cada rol tiene una página de dashboard distinta, con componentes y datos adaptados a sus necesidades:

- **AdminDashboard:** panel administrativo completo con CRUD de usuarios y cursos, estadísticas, asistencias por curso y alertas. Consume datos reales del BFF mediante el hook `useDashboardData()`.
- **TeacherAccountPage:** portal docente con vista de cursos, asistencia, anotaciones y calificaciones. También consume datos reales del BFF.
- **StudentDashboardPage:** panel del estudiante con calificaciones, asistencias y evaluaciones próximas, filtradas por su propio ID de usuario.
- **GuardianDashboardPage:** panel del apoderado con información consolidada de todos los estudiantes registrados en el sistema.

Estilos y dependencias.

El proyecto usa CSS plano con nombres de clases modulares y diseño responsivo. Las dependencias principales son React 18, react-dom, react-router-dom 6, axios 1.6 para peticiones HTTP, y Typescript 5.3 con tipado estricto habilitado en `tsconfig.json`

7. Infraestructura

ClassFlow se despliega en dos entornos de infraestructura diferenciados: local (desarrollo) y Kubernetes (producción).

7.1. Local (Docker Compose)

El archivo `docker-compose.yml` orquesta 14 contenedores en una red bridge llamada `classflow-back-net`:

Tipo	Contenedores
Bases de datos	auth-db, academic-db, assistance-db, message-db, notification-db (PostgreSQL 15 Alpine)
Microservicios	ms-auth, ms-academic, ms-assistance, ms-message, ms-notification (Spring Boot 3.5.14, JDK 25)
Infraestructura	api-gateway (Spring Cloud Gateway, puerto 8080), bff (Spring WebFlux, puerto 8086)

Frontend	frontend (Nginx, puerto 3000)
----------	-------------------------------

Cada base de datos incluye un healthcheck con `pg_isready` para que los microservicios esperen a que la base de datos esté lista antes de iniciar. Los microservicios y el frontend también tienen su propio healthcheck vía `/actuator/health`. Los volúmenes nombrados (ej. `academic-postgres-data`) aseguran persistencia de datos entre reinicios. Las variables de entorno `MAIL_USERNAME` y `MAIL_PASSWORD` se cargan desde un archivo `.env` para el servicio de notificaciones.

7.2. Kubernetes

El despliegue en Kubernetes de ClassFlow sigue una arquitectura estándar para aplicaciones basadas en microservicios. El clúster aloja un total de 12 pods distribuidos en el namespace `classflow`: 5 bases de datos PostgreSQL, 5 microservicios Spring Boot, 1 BFF, 1 API Gateway y 1 frontend Nginx. Cada microservicio se despliega como un Deployment con al menos 1 réplica, acompañado de un Service de tipo ClusterIP para la comunicación interna. Las bases de datos utilizan StatefulSet con almacenamiento persistente mediante PersistentVolumeClaim, lo que garantiza que los pods conserven su identidad y datos incluso después de reinicios.

La comunicación dentro del clúster opera de la siguiente manera: el frontend se sirve al exterior a través de un Ingress Controller (Traefik) que expone el dominio y gestiona el certificado TLS. El tráfico entrante llega al Ingress, que lo redirige al Service del API Gateway. El Gateway enruta internamente hacia los Services del BFF y de los microservicios usando los nombres DNS internos de Kubernetes (`ms-auth.namespace.svc.cluster.local`). Cada microservicio se conecta a su base de datos mediante el Service correspondiente, también resuelto por DNS interno. Las credenciales de bases de datos, la clave JWT y las variables de entorno específicas se inyectan a través de Secrets y ConfigMaps, que se montan como variables de entorno en los pods. Los health checks (`readinessProbe` y `livenessProbe`) sobre `/actuator/health` permiten a Kubernetes gestionar el tráfico y reiniciar pods que fallen.

8. Calidad y Cobertura

El proyecto incluye 44 clases de prueba distribuidas entre los 7 servicios, escritas con JUnit 5 y Mockito. Cada servicio cubre sus controladores, servicios, repositorios y componentes transversales como el manejador global de excepciones y la configuración de Swagger.

Servicio	Tests	Capas cubiertas
ms.-auth	8	Controller, Service, Repository, Security (JWT + Filtro), ExceptionHandler, SwaggerConfig
ms-academic	10	Controller (4), Service (4),

Servicio	Tests	Capas cubiertas
		Repository (1), Config
ms-assistance	6	Controller, Service, Repository, Config
ms-message	6	Controller, Service, Repository, Config
ms-notification	6	Controller, Service, Repository, ExceptionHandler, EmailService, SwaggerConfig
api-gateway	3	ExceptionHandler, CorsConfig, SwaggerConfig
bff	2	Controller, Service

Estrategia de pruebas:

Controladores:

- Se prueban con `@WebMvcTest` y `@MockitoBean` para simular los servicios, validando códigos de estado HTTP, estructura JSON y manejo de errores. Ejemplo: `AuthControllerTest` verifica login exitoso (200), credenciales inválidas (401) y registro de usuario (201).

Servicios:

- Se prueban con `@ExtendWith(MockitoExtension.class)` inyectando dependencias mock con `@Mock` e `@InjectMocks`. Se validan reglas de negocio, transformaciones entre entidades y DTOs, y excepciones esperadas.

Repositorios:

- Se prueban con `@DataJpaTest` sobre una base de datos H2 en memoria, verificando consultas derivadas y operaciones CRUD básicas.

Componentes transversales:

- `GlobalExceptionHandlerTest` verifica que las excepciones lanzadas se traduzcan en respuestas JSON coherentes; `SwaggerConfigTest` confirma que la configuración de OpenAPI se inicialice sin errores.

9. Documentación de APIs

Cada microservicio expone su documentación de API mediante Springdoc OpenAPI (`springdoc-openapi`), que genera automáticamente una interfaz Swagger UI navegable.

Servicio	Dependencia	URL
ms-auth	springdoc-openapi-starter-webmvc-ui	http://localhost:8081/swagger-ui.html
ms-academic	springdoc-openapi-starter-webmvc-ui	http://localhost:8082/swagger-ui.html
ma-assistance	springdoc-openapi-starter-webmvc-ui	http://localhost:8083/swagger-ui.html
ms-message	springdoc-openapi-starter-webmvc-ui	http://localhost:8084/swagger-ui.html
ms-notification	springdoc-openapi-starter-webmvc-ui	http://localhost:8085/swagger-ui.html
bff	springdoc-openapi-starter-webmvc-ui	http://localhost:8086/swagger-ui.html
api-gateway	springdoc-openapi-starter-webmvc-ui	http://localhost:8080/swagger-ui.html

Los servicios con Spring MVC usan la variante webmvc-ui, mientras que bff y api-gateway (reactivos con WebFlux) usan webflux-ui. La configuración se realiza mediante una clase SwaggerConfig en cada servicio, que define el título, la versión y la descripción de la API mediante el objeto OpenAPI. Adicionalmente, los endpoints incluyen anotaciones @Operation y @ApiResponse que enriquecen la documentación generada con descripciones, códigos de respuesta y ejemplos. Esto permite probar cada endpoint directamente desde el navegador sin necesidad de herramientas externas como Postman.

10. Manejo de Errores

Cada microservicio implementa un manejador global de excepciones mediante la anotación @RestControllerAdvice, lo que garantiza que todos los errores lanzados desde controladores, servicios o repositorios se capturen en un punto central y se respondan con un formato JSON estandarizado. Los 7 servicios (ms-auth, ms-academic, ms-assistance, ms-message, ms-notification, bff y api-gateway) incluyen su propia clase GlobalExceptionHandler en el paquete exception/.

Excepciones manejadas:

MethodArgumentNotValidException:

- errores de validación en los DTOs de entrada (campos obligatorios, formato de email, longitud mínima). Se devuelve un mapa con el nombre del campo y el mensaje de error correspondiente, con código 400.

RuntimeException:

- errores genéricos de negocio como "usuario no encontrado", "email ya registrado", "credenciales inválidas" o "contraseña actual incorrecta". Se devuelve un objeto con el mensaje de error y código 400.

Exception (en algunos servicios como ms-notification):

- errores no contemplados, con código 500 y mensaje genérico.

Estructura de la respuesta de error:

```
{
  "error": "User not found with ID: 999"
}
```

Para errores de validación:

```
{
  "email": "El email es obligatorio",
  "password": "La contraseña es obligatoria"
}
```

Este enfoque asegura que el frontend reciba errores predecibles y pueda mostrarlos al usuario sin importar qué capa del backend haya fallado. Cada servicio extiende o personaliza este manejador según las excepciones específicas de su dominio.

11. Convenciones y Normas

El proyecto sigue un conjunto de convenciones definidas al inicio del desarrollo, verificables en el código fuente actual:

Idioma:

- El código fuente (nombres de clases, métodos, variables, parámetros) está escrito íntegramente en inglés. Los comentarios explicativos dentro del código están redactados en español, como se aprecia en comentarios como *// Implementar según sea necesario* o *// Limpiar cookies si existen*. Esta separación permite que el código sea legible para hablantes de cualquier idioma mientras los comentarios facilitan el entendimiento del equipo.

Convención de paquetes Java:

- Todos los paquetes siguen la estructura `com.ohiggins.classflow.{artefacto}`, escrita completamente en minúsculas y sin guiones. Cada artefacto corresponde al nombre del módulo Maven sin el prefijo `ms-`

Módulo	Package base
ms-auth	com.ohiggins.classflow.auth
ms-academic	com.ohiggins.classflow.academic
ms-assistance	com.ohiggins.classflow.assistance
ms-message	com.ohiggins.classflow.message
ms-notification	com.ohiggins.classflow.notification
bff	com.ohiggins.classflow.bff
api-gateway	com.ohiggins.classflow.gateway

Convención de nombres:

- Las clases usan PascalCase (ej. AuthServiceApplication, GlobalExceptionHandler), los métodos y variables usan camelCase (ej. getDashboard(), userRepository), y las constantes usan UPPER_SNAKE_CASE (ej. ROUTES.DASHBOARD_ADMIN, LOCAL_STORAGE_KEYS.USER_TOKEN). Los nombres de las tablas en la base de datos van en plural y minúsculas (users, courses, evaluations, grades).

Migraciones Flyway:

- Siguen la nomenclatura V{version}__{descripcion}.sql, donde la versión es un número incremental y la descripción usa snake_case en inglés (ej. V1__initial_schema.sql, V2__seed_data.sql).

Frontend:

- TypeScript con modo estricto habilitado (strict: true en tsconfig.json), alias de importación con @ para rutas absolutas (@services, @components, @pages), y componentes en archivos .tsx con el mismo nombre del componente exportado.

12. Seguridad

Cuando un usuario inicia sesión con su email y contraseña, el sistema genera un token JWT, que funciona como un carnet digital. Ese token se firma con una clave secreta usando el algoritmo HS256, de modo que si alguien intenta modificarlo, el servidor lo detecta y lo rechaza. El token tiene una validez de 24 horas, después de las cuales el usuario debe volver a iniciar sesión.

ClassFlow maneja cuatro roles de usuario. El administrador tiene acceso completo al sistema y puede crear, editar o eliminar usuarios y cursos. El docente gestiona los cursos que tiene asignados, registra asistencia, calificaciones y anotaciones sobre los estudiantes. El estudiante solo puede consultar sus propias notas, asistencias y evaluaciones pendientes. El apoderado supervisa el rendimiento de los estudiantes que tiene a su cargo.

Cada rol solo puede ver las pantallas y los datos que le corresponden. El frontend verifica el rol antes de mostrar cualquier página, y si alguien intenta entrar a una página que no le corresponde, aparece una pantalla de acceso denegado.

En el entorno local, los datos viajan sin cifrar dentro de la red de Docker porque es un entorno controlado. En producción, cuando se despliega en Kubernetes, todo el tráfico que entra desde internet va cifrado con TLS (HTTPS). Además, Traefik puede validar el token JWT antes de que la petición llegue al API Gateway, actuando como una barrera adicional.

Las contraseñas nunca se guardan en texto plano. Se almacenan usando BCrypt, un algoritmo que aplica un hash y un salt aleatorio. Esto significa que aunque alguien acceda a la base de datos, no puede recuperar las contraseñas originales.

Las credenciales sensibles, como la clave JWT, las contraseñas de las bases de datos o las credenciales de correo, no están escritas en el código fuente. En el entorno local se cargan desde un archivo `.env` que está excluido del repositorio de Git. En Kubernetes se inyectan mediante objetos `Secrets` y `ConfigMaps`, que se montan como variables de entorno en los contenedores.

El token JWT se guarda en el almacenamiento local del navegador. Cada petición al backend incluye el token en el encabezado `Authorization`. Los únicos endpoints que no requieren token son los públicos, como login y registro. Cuando el usuario cierra sesión, el token y los datos del usuario se eliminan del navegador.

El API Gateway está configurado para aceptar peticiones desde cualquier origen, lo cual es necesario porque el frontend se ejecuta en un puerto diferente al del backend. Solo el gateway maneja esta configuración; los microservicios internos no la necesitan.

La inyección SQL se previene usando JPA y Hibernate con consultas parametrizadas. El CSRF está deshabilitado porque la API no mantiene sesiones del lado del servidor, sino que usa tokens JWT. Las contraseñas están protegidas con BCrypt. El robo de sesión se dificulta porque el token está firmado y expira a las 24 horas. Las rutas del frontend están protegidas por rol. En producción, los datos viajan cifrados con TLS, y las credenciales sensibles nunca quedan expuestas en el repositorio porque se gestionan con `.env` y `Secrets` de Kubernetes.

13. Seguimiento de Errores

GlitchTip es una plataforma de código abierto que permite centralizar y visualizar los errores de todos los microservicios en un solo lugar. Funciona de manera similar a Sentry, pero se puede instalar en los propios servidores del proyecto sin depender de servicios externos.

En el archivo de Docker Compose se incluyen tres contenedores para GlitchTip: una base de datos PostgreSQL exclusiva, un servicio de caché llamado Valkey, y la aplicación principal que se despliega en el puerto 8000. Al arrancar el proyecto por primera vez, hay que entrar a `http://localhost:8000`, crear una cuenta y un proyecto, y copiar el DSN que

entrega la plataforma. Ese DSN se pega en el archivo .env del backend en la variable SENTRY_DSN. Mientras esa variable esté vacía, el SDK no hace nada y los errores no se reportan.

Todos los microservicios, incluyendo el API Gateway y el BFF, incluyen la dependencia de Sentry en su archivo pom.xml. Cuando ocurre una excepción, el GlobalExceptionHandler de cada servicio la captura y la envía automáticamente a GlitchTip mediante `Sentry.captureException(ex)`. Esto permite tener un historial centralizado de todos los errores que ocurren en el sistema.

Para el registro de eventos, cada microservicio usa SLF4J con Logback, que es la configuración que trae Spring Boot por defecto. Los servicios importantes usan la anotación `@Slf4j` de Lombok para tener un logger disponible sin escribir código repetitivo.

Los niveles de registro varían según el servicio. El API Gateway tiene configurado nivel DEBUG para la puerta de enlace y TRACE para el enrutamiento, lo que ayuda a depurar problemas de conexión entre servicios. En los archivos de prueba, el nivel se sube a WARN para reducir el ruido en la salida de los tests.

Cuando se ejecutan los microservicios con el perfil de Docker, el logging se ajusta para producir salida estructurada, lo que facilita su procesamiento si más adelante se integra con herramientas de agregación de logs.

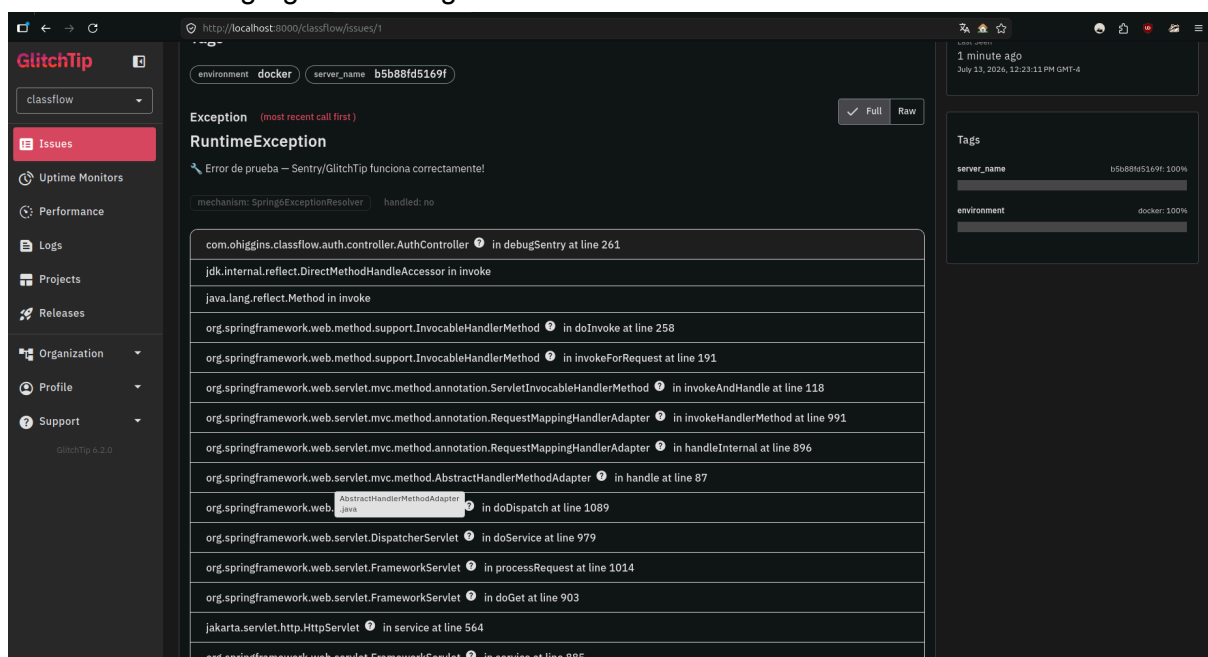


Fig 4. GlitchTip en ejecución. Para este ejemplo se ha inducido a un error en el sistema dentro del microservicio ms-auth a modo de comprobación de su funcionamiento óptimo. Se muestra el backtrace del error del sistema.

14. Anexos

Enlace al proyecto en GitHub:

<https://github.com/theseventhshadow/ClassFlow>

En el repositorio encontrará más información respecto al proyecto.